MIVA OPEN UNIVERSITY

Project Report & Defence Documentation

Course: COS 106 — Introduction to Web Technologies

Multi-Page Personal Portfolio & Interactive Academic Web Application

Jehoiada Enyinyigocho Abu
Bachelor of Software Engineering
Student Repository: github.com/jehoabu/cos_106_portfolio_project

Academic Year 2026

Abstract

This document serves as a comprehensive academic project report and oral defence presentation guide for a multi-page web portfolio and interactive academic web application developed for evaluation in **COS 106: Introduction to Web Technologies** at Miva Open University. Built using semantic HTML5, modern CSS3 layout systems (Flexbox, CSS Grid, and Glassmorphism design patterns), and vanilla JavaScript ES6+, the project demonstrates strict adherence to web accessibility standards (WCAG), Cross-Site Scripting (XSS) defense, event-driven DOM manipulation, and dynamic client-side input validation. This report provides complete architectural breakdowns, UI screenshot walkthroughs, JavaScript implementation analyses, code snippet explanations, and anticipated defence Q&A modules.

Contents

1 Project Overview **3**
1.1 Introduction . . . 3
1.2 Purpose and Objectives . . . 3
1.3 Major Features and Functionalities . . . 3
1.4 Technologies Used . . . 3

2 Screenshots and Page Layout Explanations **4**
2.1 Homepage & Single-Pass Typewriter Banner (`index.html`) . . . 4
2.2 About Page & Academic Qualifications Table (`about.html`) . . . 4
2.3 Projects Gallery & Responsive Grid (`projects.html`) . . . 5
2.4 Academic Planner Interface (`planner.html`) . . . 5
2.5 Contact Portal & Alert Banners (`contact.html`) . . . 6

3 JavaScript Concepts & Implementation Analysis **7**
3.1 Single-Pass Character Typewriter Animation (`js/typing.js`) . . . 7
3.2 Cross-Site Scripting (XSS) Mitigation . . . 7
3.3 Event Delegation Pattern . . . 7
3.4 Regular Expression Validation & Trimming . . . 8
3.5 Separation of Concerns: Class-Based Feedback . . . 8

4 Codebase Breakdown & Defence Preparation **9**
4.1 Single-Pass Typewriter Script (`js/typing.js`) 9
4.2 Secure Academic Planner Logic (`js/planner.js`) 9
4.3 Contact Form Validation Logic (`js/contact.js`) 10
4.4 Anticipated Defence Questions & Answers 12

5 Project Links & Deployment Information **13**
5.1 Public Access URLs 13
5.2 Deployment Pipeline 13
5.3 Verification Summary 13

1 Project Overview

1.1 Introduction

The project is a responsive, accessible, multi-page personal portfolio integrated with client-side interactive tools. Engineered from the ground up, the application serves a dual purpose: establishing a professional digital presence for a Software Engineering undergraduate and demonstrating technical proficiency in client-side web development standards as specified in the COS 106 curriculum.

1.2 Purpose and Objectives

The core objectives guiding the design and development of this web application are:

1. **Demonstrate Mastery of Core Web Technologies:** Build a complete, functional website utilizing pure HTML5, CSS3, and ES6+ JavaScript without relying on heavy external libraries or frameworks.
2. **Deliver Interactive Student Utilities:** Create an Academic Task Planner to allow students to organize coursework deadlines dynamically in real-time.
3. **Implement Security and Accessibility Standards:** Neutralize Cross-Site Scripting (XSS) threats using safe DOM manipulation, enforce WCAG keyboard focus states, and incorporate screen-reader ARIA live alert regions.
4. **Execute Modern UI/UX Architecture:** Apply modern design standards including sticky glassmorphism navigation headers, auto-fitting CSS Grid layouts, and custom keyframe typing animations.

1.3 Major Features and Functionalities

The web application consists of five interlinked HTML5 pages and dedicated script/style modules:

- **Homepage (`index.html` & `js/typing.js`):** Houses the hero presentation, circular avatar mask with ring highlight, dynamic single-pass typewriter welcome message, and structured biography box.
- **About Page (`about.html`):** Contains an academic qualification table formatted with semantic `<thead>`/`<tbody>` tags and a flexbox technical skills tag cloud.
- **Projects Gallery (`projects.html`):** Uses auto-responsive CSS Grid columns (`repeat(auto-fit, minmax(280px, 1fr))`), project cards with hover-lift micro-interactions, and embedded HTML5 video controls.
- **Academic Planner (`planner.html` & `js/planner.js`):** Provides client-side task creation, completion toggling, and task deletion built on Event Delegation and secure text node assignment.
- **Contact Portal (`contact.html` & `js/contact.js`):** Executes real-time string trimming, email RegEx verification, numeric phone validation, and dynamic ARIA feedback alert states.

1.4 Technologies Used

Table 1 summarizes the technologies and frameworks comprising the codebase.

Table 1: System Architecture & Technology Stack

Layer	Technology / Module	Core Purpose
Structure	HTML5 Markup	Semantic layout tags, tables, forms, accessibility attributes
Design System	CSS3 (Custom Properties)	Glassmorphism, CSS Grid, Flexbox, animations, focus rings
Typography	Google Montserrat Font	Modern, highly legibile sans-serif font family.
Behavior	Vanilla JavaScript ES6+	DOM manipulation, typing effect, event delegation, RegEx.
Version Control	Git & GitHub	Distributed source control and repository management.
Hosting	GitHub Pages	Production static hosting and automated deployment.
Documentation	LaTeX (`report.tex`)	Academic documentation and defence presentation prep.

2 Screenshots and Page Layout Explanations

This section details each major section of the portfolio, explaining its visual layout, functional purpose, and technical implementation. (Frame placeholders represent the exact screenshot locations for defence slide integration).

2.1 Homepage & Single-Pass Typewriter Banner (`index.html`)

[Screenshot Placeholder: Homepage & Hero Section]
Showing Sticky Glassmorphism Header, Circular Avatar Ring, Single-Pass Typewriter Animation, and Biography Card.

Figure 1: Homepage UI layout showcasing avatar styling and dynamic welcome message.

- **Section Description:** The homepage serves as the entry point for visitors. It displays a top sticky navigation bar, a highlighted portrait photo, a dynamic welcome message, and a biography card outlining educational goals.
- **Purpose:** Establishes personal branding and provides immediate context about the author's background in Software Engineering.
- **Implementation Details:** The image is styled with `border-radius: 50%`, `object-fit: cover`, and a dual ring drop-shadow box. The welcome message container hosts a `<span id="typing-text">` element populated character-by-character by `js/typing.js`, flanked by a blinking keyframe cursor (`.typing-cursor`).

2.2 About Page & Academic Qualifications Table (`about.html`)

- **Section Description:** Details the author's formal education timeline, career aspirations, and core technical skills.
- **Purpose:** Demonstrates mastery of complex HTML data tabular structures and semantic organization.

[Screenshot Placeholder: About Page & Qualifications Table]
Displaying Semantic Education Table with Alternating Shading and Flexbox Skill Badges.

Figure 2: About page layout displaying structured academic table and skill tags.

- **Implementation Details:** Built using `<table>`, `<thead>`, and `<tbody>`. Alternating rows are styled using `tr:nth-child(even)` for enhanced legibility. Technical skills (`<ul class="skills">`) use Flexbox with `flex-wrap: wrap` to render visual pill tags.

2.3 Projects Gallery & Responsive Grid (`projects.html`)

[Screenshot Placeholder: Projects Gallery]
Showing Auto-Fitting Grid Columns, Card Hover Elevation, and Embedded Video Controls.

Figure 3: Projects showcase featuring auto-responsive CSS Grid layout.

- **Section Description:** Highlights practical projects built during coursework, complete with descriptions, tech tags, action links, and a video demonstration.
- **Purpose:** Demonstrates multimedia integration and advanced 2D CSS Grid responsiveness.
- **Implementation Details:** Formatted via `display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr))`. Cards lift smoothly on hover via `transform: translateY(-6px)`. Contains a fully compliant HTML5 `<video>` element with custom control attributes.

2.4 Academic Planner Interface (`planner.html`)

- **Section Description:** An interactive task manager where users can enter tasks, pick due dates, mark items complete, or delete entries.
- **Purpose:** Satisfies the requirement for dynamic client-side DOM creation and manipulation.
- **Implementation Details:** Tasks are rendered as `<li>` items inside `<ul id="task-list">`. Completed items dynamically acquire the `.completed` class, applying strike-through styling and opacity reduction.

[Screenshot Placeholder: Academic Planner Application]
Illustrating Input Controls, Task Rows, Completion Checkbox Toggles, and Delete Action Buttons.

Figure 4: Interactive Academic Planner web application.

[Screenshot Placeholder: Contact Portal & Feedback Alerts]
Displaying Input Fields, Explicit Focus Outlines, and Dynamic Success/Error Banners.

Figure 5: Contact page illustrating form validation feedback states.

2.5 Contact Portal & Alert Banners (`contact.html`)

- **Section Description:** A user inquiry form featuring real-time client validation and accessible status reporting.
- **Purpose:** Demonstrates advanced client-side form validation, RegEx pattern matching, and accessible alert design.
- **Implementation Details:** Inputs utilize explicit focus rings (`outline: 2px solid #1098AD`). Validation updates populate a dynamic banner configured with `role="alert"` for immediate screen-reader announcement.

3 JavaScript Concepts & Implementation Analysis

This section outlines the architectural patterns and JavaScript ES6+ features implemented across the application's script files (`typing.js`, `planner.js`, and `contact.js`).

3.1 Single-Pass Character Typewriter Animation (`js/typing.js`)

To provide an engaging landing experience without distracting repeating loops, a typewriter script types the welcome message once upon DOM load:

- **DOM Initialization:** Listens for `DOMContentLoaded` to prevent execution prior to node availability.
- **Defensive Guard Clause:** Incorporates `if (!typingElement) return;` to avoid console error throwing on pages lacking the target span.
- **Recursive Recursion:** Utilizes a recursive `typeWriter()` function driven by `setTimeout(typeWriter, typingSpeed)` to append characters via `.charAt(charIndex)`.
- **Graceful Cursor Fade:** Once typing completes, the blinking pipe cursor (`|`) remains visible briefly before fading out smoothly via CSS `transition: opacity 1s ease`.

3.2 Cross-Site Scripting (XSS) Mitigation

In `planner.js`, malicious script injection is prevented by strictly avoiding `innerHTML` when rendering user input:

- **Vulnerability Analysis:** Assigning strings directly to `innerHTML` forces the browser HTML parser to compile input. If a user inputs `<img src=x onerror=alert(1)>`, the script executes.
- **Defensive Strategy:** The planner creates an explicit node via `document.createElement('span')` and assigns input text via `textSpan.textContent = ...`. The `.textContent` property instructs the engine to treat string data purely as literal text nodes, rendering HTML tags harmlessly.

3.3 Event Delegation Pattern

To optimize memory usage and support dynamically generated DOM elements, `planner.js` employs **Event Delegation**:

- Rather than binding individual event listeners to every generated task button or checkbox, a single event listener is attached to the parent container (`<ul id="task-list">`).
- Upon a click event, the handler evaluates `event.target.tagName` or `event.target.type`.
- If a button is clicked, `event.target.parentElement.remove()` deletes the target item.
- If a checkbox is clicked, `listItem.classList.toggle('completed')` updates state instantly.

3.4 Regular Expression Validation & Trimming

In `contact.js`, inputs undergo sanitization and format verification prior to acceptance:

1. **String Trimming:** `.trim()` strips leading and trailing whitespace to block blank-space bypasses.
2. **Email Pattern Matching:** Evaluated against `/^[^\s@]+@[^\s@]+\.[^\s@]+$/` to verify standard structure.
3. **Phone Digit Matching:** Evaluated against `/^\d+$/` to enforce purely numeric sequences.

3.5 Separation of Concerns: Class-Based Feedback

In alignment with modular software design, JavaScript does not apply inline styling directly for validation states. Instead, helper functions assign styled CSS classes:

- Errors set `messageDisplay.className = 'alert alert-error'`.
- Success sets `messageDisplay.className = 'alert alert-success'`.

4 Codebase Breakdown & Defence Preparation

This section highlights critical listings from the project repository, accompanied by explanation points for oral defence.

4.1 Single-Pass Typewriter Script (`js/typing.js`)

```
document.addEventListener('DOMContentLoaded', () => {
  const typingElement = document.getElementById('typing-text');
  const cursorElement = document.querySelector('.typing-cursor');

  if (!typingElement) return;

  const textToType = "Software Engineering student at Miva Open University, passionate
    about technology, creativity, innovation, and problem-solving.";
  let charIndex = 0;
  const typingSpeed = 40;

  function typeWriter() {
    if (charIndex < textToType.length) {
      typingElement.textContent += textToType.charAt(charIndex);
      charIndex++;
      setTimeout(typeWriter, typingSpeed);
    } else {
      if (cursorElement) {
        setTimeout(() => {
          cursorElement.style.transition = 'opacity 1s ease';
          cursorElement.style.opacity = '0';
          setTimeout(() => { cursorElement.style.display = 'none'; }, 1000);
        }, 2500);
      }
    }
  }

  typingElement.textContent = '';
  typeWriter();
});
```

Listing 1: Hero Section Typewriter Animation

Key Defence Explanation Points:

- **Why use `DOMContentLoaded`?** Ensures the script executes only after HTML parsing completes, guaranteeing `document.getElementById('typing-text')` finds a valid DOM node.
- **Why recursion over `setInterval`?** Recursive `setTimeout` calls offer finer control, allowing timing adjustments per character and clean termination without managing interval IDs.

4.2 Secure Academic Planner Logic (`js/planner.js`)

```
const taskForm = document.getElementById('task');
const taskInput = document.getElementById('task_input');
const deadlineInput = document.getElementById('deadline');
const taskList = document.getElementById('task-list');

taskForm.addEventListener('submit', function(event) {
    event.preventDefault();

```

```
    const taskText = taskInput.value.trim();
    const deadlineText = deadlineInput.value.trim();

    if (!taskText) return;

    const newListItem = document.createElement('li');

    // Secure node assignment against XSS
    const textSpan = document.createElement('span');
    textSpan.textContent = `${taskText} (Due: ${deadlineText}) `;

    const checkbox = document.createElement('input');
    checkbox.type = 'checkbox';

    const deleteBtn = document.createElement('button');
    deleteBtn.type = 'button';
    deleteBtn.textContent = 'Delete';

    newListItem.appendChild(textSpan);
    newListItem.appendChild(checkbox);
    newListItem.appendChild(deleteBtn);

    taskList.appendChild(newListItem);

    taskInput.value = '';
    deadlineInput.value = '';
});

// Event Delegation on parent UL
taskList.addEventListener('click', function(event) {
    if (event.target.tagName === 'BUTTON') {
        event.target.parentElement.remove();
    }
    if (event.target.type === 'checkbox') {
        const listItem = event.target.parentElement;
        listItem.classList.toggle('completed');
    }
});
```

Listing 2: Task List Creation and Event Delegation

Key Defence Explanation Points:

- **Why use `event.preventDefault()`?** Standard HTML form submission sends an HTTP request and reloads the page. Calling `preventDefault()` suppresses this, allowing JavaScript to handle task insertion dynamically.
- **How does Event Delegation work here?** By placing a single listener on `taskList`, clicks on dynamically added child buttons bubble up to the parent, eliminating the need to attach listeners to each new task individually.

4.3 Contact Form Validation Logic (`js/contact.js`)

```
const contactForm = document.getElementById('contact-form');

const messageDisplay = document.createElement('div');
messageDisplay.setAttribute('role', 'alert');
messageDisplay.className = 'alert alert-hidden';
contactForm.prepend(messageDisplay);

```

```
contactForm.addEventListener('submit', function(event) {
    event.preventDefault();

    messageDisplay.className = 'alert alert-hidden';
    messageDisplay.textContent = '';

    const name = document.getElementById('name').value.trim();
    const email = document.getElementById('email').value.trim();
    const phone = document.getElementById('phonenumber').value.trim();
    const message = document.getElementById('message').value.trim();

    if (!name || !email || !phone || !message) {
        showError("Please fill out all fields before submitting.");
        return;
    }

    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailPattern.test(email)) {
        showError("Please enter a valid email address (e.g., name@example.com).");
        return;
    }

    const phonePattern = /^\d+$/;
    if (!phonePattern.test(phone)) {
        showError("Your phone number must contain only digits (no spaces or dashes).");
        return;
    }

    showSuccess(`Thank you, ${name}! Your message has been sent successfully.`);
    contactForm.reset();
});

function showError(errorText) {
    messageDisplay.textContent = errorText;
    messageDisplay.className = 'alert alert-error';
}

function showSuccess(successText) {
    messageDisplay.textContent = successText;
    messageDisplay.className = 'alert alert-success';
}
```

Listing 3: Form Validation and Accessible ARIA Alerts

Key Defence Explanation Points:

- **Why set `role="alert"`?** Creates an ARIA live region ensuring screen-reader users receive immediate voice feedback whenever an error or success message appears.
- **What is the benefit of `.className` switching?** Decouples styling from JavaScript logic. Visual updates remain governed by `forms.css`.

4.4 Anticipated Defence Questions & Answers

Defence Q&A 1: Prevention of XSS Vulnerabilities

Question: How does your portfolio prevent Cross-Site Scripting (XSS) when users submit task entries?
Ideal Answer: We avoid using `innerHTML` for user-supplied data. Instead, we generate element nodes via `document.createElement('span')` and assign input strings using `.textContent`. This forces the browser rendering engine to treat input strictly as raw text characters, preventing malicious tags like `<script>` from compiling or executing.

Defence Q&A 2: CSS Grid Responsiveness

Question: How does your projects gallery adapt to mobile screens without complex media queries?
Ideal Answer: We utilize the CSS Grid auto-fit rule: `grid-template-columns: repeat(auto-fit, minmax(280px, 1fr))`. This instructs the browser to fill available space with equal fractional units (`1fr`) while enforcing a strict minimum card width of `280px`. When the viewport contracts below this width, cards automatically wrap to the next line.

Defence Q&A 3: Performance Optimization via Event Delegation

Question: Why did you place the click listener on the parent `<ul>` in your planner instead of individual buttons?
Ideal Answer: Attaching event listeners to individual items is inefficient and fails for dynamically added elements. By leveraging event bubbling via Event Delegation, a single listener on the parent `<ul>` handles click events for all existing and future tasks efficiently.

5 Project Links & Deployment Information

The portfolio application is deployed live on GitHub Pages and maintained under version control.

5.1 Public Access URLs

- **Live Hosted Website:**
  https://jehoabu.github.io/cos_106_portfolio_project/
- **GitHub Source Repository:**
  https://github.com/jehoabu/cos_106_portfolio_project

5.2 Deployment Pipeline

The application is deployed using **GitHub Pages**:

1. Local commits are pushed to the `main` branch on GitHub.
2. GitHub Pages detects changes to the root folder.
3. An automated workflow builds and deploys static assets to a global CDN.

5.3 Verification Summary

The deployed site has been verified across Chromium, WebKit, and Gecko engines to guarantee responsiveness, functional scripts, accessibility standards, and valid markup compliance.